

Backend & System Architecture — Safe AI Automation for MSP Operations

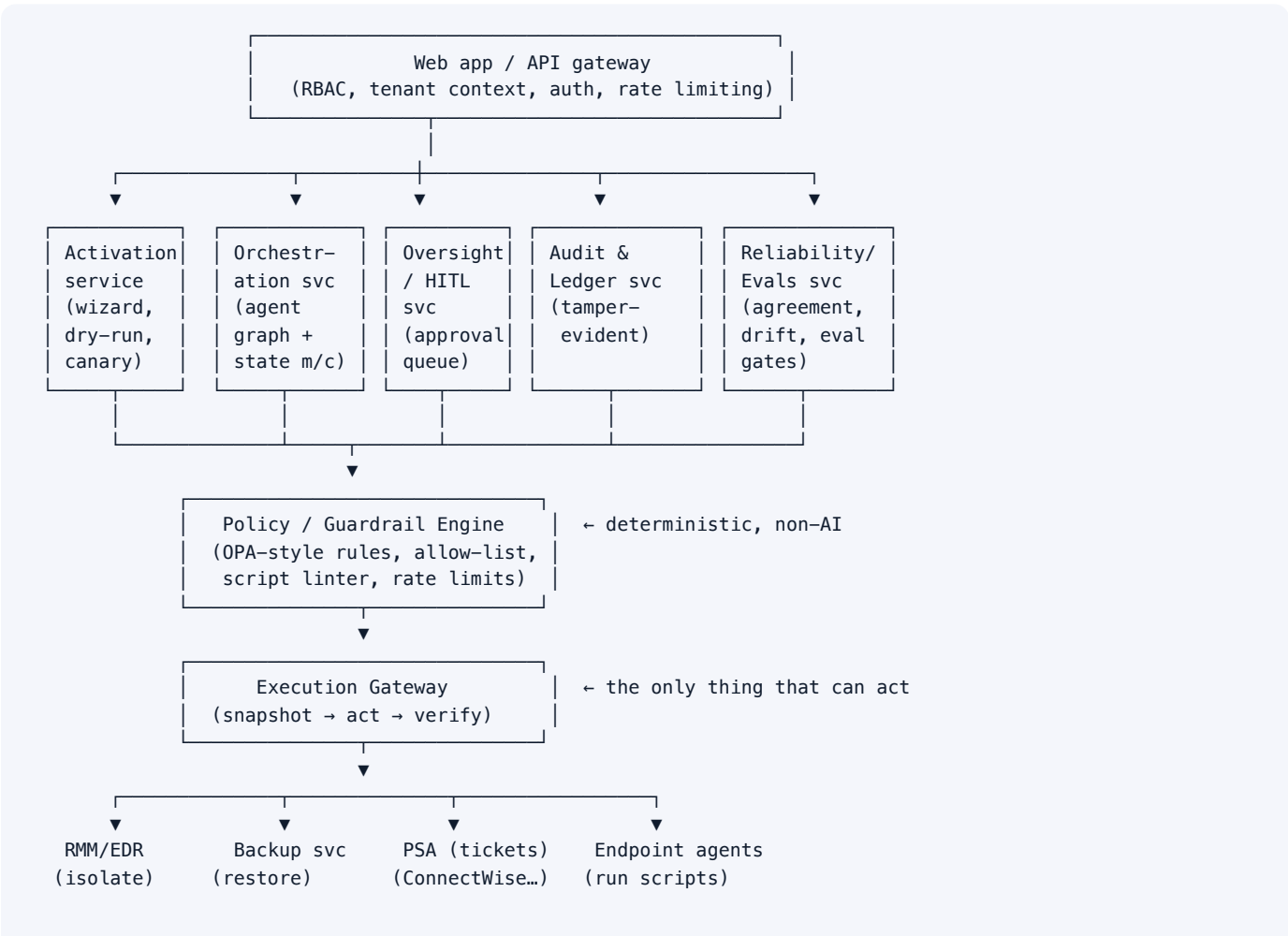
Concept: *The Trust Ladder* · **Author:** Munshi Javed Akhtar · **Version:** 1.0 **Audience:** Engineering, architecture, security, and data-science reviewers **Companion artifacts:** PRD (PRD_Safe_AI_Automation.md), prototype (prototype/index.html), one-page briefing

This document describes *how the system works under the hood* — the services, the agent-orchestration model, the guardrail enforcement, the execution/rollback path, the audit ledger, and the security posture. It is deliberately implementation-flavoured so it can be discussed with engineers, not just product stakeholders.

1. Architectural principles (what the design optimises for)

- The model proposes; the platform disposes.** The LLM/agent layer only ever *emits a proposed action*. Whether that action can execute is decided by deterministic, non-AI services (policy engine, RBAC, execution guard). No LLM output is ever trusted as an authorisation.
- Everything is an auditable, reversible transaction.** Actions are not fire-and-forget API calls; they are recorded, snapshotted, and undoable.
- Tenant isolation is a hard boundary, not a filter.** Multi-tenancy is enforced at the data, credential, and execution layers — never only in application code.
- Stateless compute, durable state.** Agents and workers are stateless and horizontally scalable; all durable state (run status, approvals, ledger) lives in databases and a workflow engine so any node can resume any run.
- Fail closed.** On ambiguity, timeout, policy error, or component failure, the system escalates to a human or halts — it never “acts anyway.”

2. High-level component map



3. Agent orchestration

3.1 Model: a directed graph with a durable state machine

Orchestration is a **stateful graph** (implementable with LangGraph, Temporal, or a custom state machine on a durable workflow engine). Each node is an agent step or a deterministic gate; edges are transitions. The graph is **persisted per run** so it survives restarts and can be resumed, replayed, and audited.

For the ransomware workflow:

```
[trigger] → (Triage Agent) → {classification?}
                                     | true positive → (Remediation Agent) → [policy gate] → [HITL gate] →
[execute] → [verify] → [close]         | ambiguous      → [HITL: human classifies]
                                     | false positive → [close + log]
```

3.2 Agent roles are separate, scoped services

- **Triage Agent** — read-only tools (query EDR logs, backup catalog, asset tags). *Cannot* propose actions that mutate state. Its output is a structured Classification object.
- **Remediation Agent** — may *propose* mutating actions, emitted as a structured ActionPlan (never raw side-effects).

Separation matters for security: the agent with the dangerous tools is the smaller, more constrained one, and it only runs on a validated hand-off.

3.3 Tool use is mediated, never direct

Agents never call RMM/backup APIs directly. They call a **Tool Broker** that: - exposes only the tools allowed for that agent + tenant + autonomy rung, - injects **short-lived, least-privilege, tenant-scoped credentials** (agents never see long-lived secrets), - validates tool arguments against a schema before the call, - logs every tool invocation to the ledger.

3.4 Structured outputs, not free text

Every agent step returns a **validated JSON object** (schema-enforced; reject-and-retry on mismatch). Free-form model text is never parsed for control flow. This is what makes downstream gates deterministic.

3.5 Autonomy rung as a runtime parameter

The Trust Ladder rung (L0–L3) is stored per (agent, tenant) and passed into the graph. It controls the transition after the policy gate: - **L0 Shadow**: stop after ActionPlan; record, don't execute. - **L1 Suggest**: create a draft in the tech queue; no execution path. - **L2 Approve-to-act**: require a HITL approval before execute. - **L3 Auto+notify**: execute if policy passes; notify after.

4. Guardrail / Policy Engine (the safety core)

This is the most important non-AI service. It sits between *proposed action* and *execution* and is **deterministic and independently testable**.

4.1 What it enforces

- **Policy rules** (OPA/Rego-style or a rules DSL): e.g. `deny if target.tag == "role:critical-infra", deny if backup.age > 24h, deny if action.reversible == false`.
- **Script/command allow-list (AST linter)**: the actual command an agent wants to run is parsed and every cmdlet/binary checked against a certified allow-list. Remove-Item, Format-Volume, registry edits, etc. are rejected regardless of model intent.
- **Rate & scope limits**: e.g. max 5 isolations/hour/tenant; action target must be inside the activated scope.
- **Prompt-injection containment**: ticket/user-supplied text is tagged as *data*, never merged into the instruction channel; actions derived from untrusted text are blocked.

4.2 Properties

- **Fail-closed:** if the policy engine errors or is unreachable, the action is denied.
 - **Versioned & signed:** every policy set has a version; the version that evaluated an action is recorded in the ledger.
 - **Testable in isolation:** policies are unit-tested and run in the eval suite (see §8) — they don't depend on model behaviour.
-

5. Execution Gateway (the only component that can change the world)

No other service can mutate an endpoint, backup, or ticket. It executes the **snapshot → act → verify** transaction:

1. **Pre-flight:** re-check policy + RBAC authorisation token at execution time (defence in depth — approval alone isn't enough).
2. **Snapshot:** capture pre-action state (e.g. volume snapshot / service state / config) → this is the rollback point.
3. **Act:** perform the action via the relevant integration (RMM isolate, backup restore, script via endpoint agent) using short-lived scoped credentials.
4. **Verify:** health-check the result (e.g. endpoint reachable, backup restored, files present). On failure → auto-rollback + escalate.
5. **Record:** write the full transaction (inputs, snapshot id, result, hashes) to the ledger.

Fleet actions (many endpoints) run in **staged batches** with health checks between batches, so a bad action can't hit the whole set at once.

Rollback is a first-class operation: given a run id, restore the recorded snapshot(s).

6. Human-in-the-loop (HITL) / Oversight service

- Maintains a **durable approval queue** per tenant. A run at L2 parks at the HITL gate and emits an approval request.
 - The approval payload = the ActionPlan + agent reasoning + evidence + per-guardrail results + confidence. (This is what the Oversight Inbox renders.)
 - **Time-boxed:** an SLA timer per request; on expiry → escalate to on-call + PSA. A live threat is never silently dropped.
 - Approve/reject/escalate decisions are signed, attributed to a human identity, and recorded. **Rejections capture a reason** that is emitted to the evals pipeline as a labelled case.
 - Enforces RBAC: only roles with approve can action items; others get read-only.
-

7. Audit & Ledger service (accountability)

- **Append-only, tamper-evident log.** Each entry is hash-chained (entry N includes hash of N-1); entries are signed. Optionally anchored to a WORM store / external notary for regulatory strength.
 - Records, per action: agent + agent version, **model + prompt version**, policy version, evidence references, human authoriser, snapshot id, reversibility flag, result, timestamps.
 - Captures the **Triage → Remediation hand-off** as linked entries (auditable multi-agent orchestration).
 - **Exportable** as a signed report for a client or auditor (SOC 2 / ISO 27001 / HIPAA contexts).
 - Read access is itself RBAC-gated and logged.
-

8. Reliability & Evals pipeline

- **Offline eval gate (pre-production):** every prompt/model/policy change runs against a **golden set** of labelled historical incidents. Gates: classification accuracy, destructive-action safety (0 bad isolations), prompt-injection resistance, policy correctness. A change can't ship if it regresses. (This also powers the activation **dry-run** — the same replay engine over a tenant's 90-day history.)
 - **Online metrics (production):** human-agreement rate, confidence calibration, guardrail-breach count (target 0), time-to-containment.
 - **Drift detection:** statistical monitors on confidence/agreement per step; breach → alert + optional auto-demote/pause of the agent's rung.
 - **Closed loop:** HITL rejections + dry-run divergences are written back as new labelled golden-set cases. Every human correction hardens the next release.
-

9. Security architecture (called out explicitly)

9.1 Identity, access, tenancy

- **RBAC** enforced at the API gateway *and* re-checked at the execution gateway (never trust a single choke point). Roles: Owner / Admin / Technician / Auditor (see PRD §6.5).
- **Multi-tenant isolation:** tenant id is part of every credential, DB row (row-level security), and execution scope. Cross-tenant access is structurally impossible, not merely filtered.
- **Least privilege for agents:** agents receive **short-lived, tenant + action-scoped tokens** from a secrets vault (e.g. Vault / cloud KMS). No agent holds standing credentials to a client estate.

9.2 LLM-specific threats

- **Prompt injection:** untrusted content (ticket text, file contents, alert descriptions) is isolated in a data channel and never concatenated into the system/instruction prompt. Any action traceable to injected text is denied by the policy engine. Injection resistance is an eval gate.
- **Excessive agency:** mitigated by the “propose vs. dispose” split — the model cannot execute; only the policy-gated execution gateway can.
- **Sensitive-data handling:** PII/hostnames are minimised and, where telemetry is shared for global model improvement, scrubbed before leaving the tenant boundary.
- **Model gateway:** all model calls go through a gateway that enforces provider allow-listing, logging, cost/rate limits, and (where required) region/data-residency routing.

9.3 Action safety

- **Allow-listing over blocklisting** for executable commands (default-deny).
- **Fail-closed** everywhere: policy errors, timeouts, and unreachable dependencies all resolve to “don’t act.”
- **Kill switch:** a control-plane flag (global for Admin/Owner, scoped for Technician) that the orchestration and execution services check before any action; engaging it freezes new executions and in-flight approvals.

9.4 Supply chain & platform

- Signed policy sets, prompt versions, and agent releases; provenance recorded in the ledger.
- Standard controls: encryption in transit/at rest, secret rotation, network segmentation between the (untrusted-input-handling) agent layer and the (privileged) execution layer.

10. Core data model (key entities)

Entity	Purpose (key fields)
Tenant	Client boundary; risk tier, compliance flags
AgentDeployment	(agent, tenant) → autonomy rung, scope, active guardrail set, status
WorkflowRun	One execution of the graph; state, current node, timestamps
Classification	Triage output; verdict, confidence, evidence refs
ActionPlan	Remediation output; proposed actions, targets, reversibility
PolicyEvaluation	Result of the guardrail engine; policy version, per-rule pass/fail
ApprovalRequest	HITL item; payload, SLA timer, decision, human id, reason
ExecutionRecord	snapshot id, action result, verify status, rollback ref
LedgerEntry	Hash-chained, signed audit record linking all of the above
EvalCase	Labelled golden-set case (incl. ones fed back from rejections)

11. Key flows

11.1 End-to-end (L2, ransomware)

```
EDR alert → event bus → Orchestration starts WorkflowRun
→ Triage Agent (read-only tools) → Classification{true_positive, conf 0.97}
→ Remediation Agent → ActionPlan{isolate + restore, reversible}
→ Policy Engine → PolicyEvaluation{4/4 pass}
→ HITL gate (L2) → ApprovalRequest → human approves (signed)
→ Execution Gateway: snapshot → isolate → restore → verify OK
→ Ledger: hash-chained entries (triage, plan, policy, approval, execution)
→ Close ticket in PSA · notify client
```

11.2 Fail-closed example (the domain-controller case)

```
Remediation ActionPlan{isolate SRV-DC-01} + conf 0.62
→ Policy Engine: deny (target.tag == role:critical-infra) AND deny (conf < 0.80)
→ No execution path. Escalate to human via HITL. Ledger records the block.
```

11.3 Rollback

```
Human/Admin triggers rollback(run_id)
→ Execution Gateway loads ExecutionRecord.snapshot_id
→ restores pre-action state → verify → Ledger append (rollback entry)
```

12. Reliability, scale & failure modes

- **Durable workflow engine** (Temporal-style) → runs survive process/node failure and resume exactly where they parked (e.g. at a HITL gate for hours).
- **Idempotent execution** keyed by run id → a retry never double-acts.
- **Backpressure & rate limits** at trigger ingestion so an alert storm can't overwhelm the system (or a tenant).
- **Degraded modes:** if the model gateway is down → agents can't propose → runs park and escalate (fail-closed), existing approvals still executable by humans. If the policy engine is down → all execution denied.
- **Observability:** traces per run, metrics per gate, and the ledger as the source of truth for "what happened."

13. Illustrative technology choices

(choices, not mandates — the design is stack-agnostic)

- **Orchestration:** Temporal or LangGraph over a durable store.
- **Policy engine:** Open Policy Agent (Rego) + a small AST linter for scripts.
- **Model access:** a model gateway (provider-agnostic; e.g. Anthropic Claude / Azure OpenAI) with logging and residency routing.
- **State/data:** Postgres (row-level security for tenancy) + object store for snapshots/exports.
- **Eventing:** Kafka/NATS event bus for triggers and cross-service events.
- **Secrets:** Vault / cloud KMS with short-lived tokens.
- **Runtime:** containerised microservices on Kubernetes; stateless agents/workers.
- **Ledger:** append-only, hash-chained table + optional WORM/notary anchor.

14. Open architectural questions

- Build orchestration on Temporal (durability, replay) vs. LangGraph (agent-native ergonomics) — or LangGraph on Temporal?
- Where exactly is the tenant trust boundary for shared/global evals — scrub-at-source vs. tenant-side aggregation only?

- Snapshot cost/retention policy for reversibility vs. storage footprint at fleet scale.
- Real-time vs. near-real-time for the audit anchor (inline notarisation adds latency to the action path).
- Legal locus of accountability at L3 (approver vs. promoter vs. MSP) — drives how much attribution metadata the ledger must retain.